

The Total-Work Cost of Bounded Retry Guarantees

ANONYMOUS AUTHOR(S)

Bounding retries can move computation under protection; reducing protected work can instead increase total work. We characterize that tradeoff for persistent whole-kernel jobs with specified completion and observation operations. With a supplied write bound and arbitrary retry allowance, independent jobs admit a complete polynomial total/protected-work frontier. If retries cover the write bound, the same frontier holds on every dependency DAG without observing cached comparisons. With no spare call, precedence instead determines the one-write frontier through Lawler’s scheduling objective. One policy with optimal protection at every hidden budget pays a larger total-work cost. A domination proof covers causal programs with free inspections and retained preparations, yielding a finite Pareto construction for all retry allowances. Hiding comparisons has an exact cost at zero retry slack. Java wrappers realize the contract, including actual tree bins and both cheap publication operations. Finite checks corroborate the formulas. Roslyn checks preserve selected source behavior and counts, while arrival timings remain mixed. The results identify exact resource tradeoffs and the interface conditions under which they hold.

CCS Concepts: • **Software and its engineering** → **Concurrent programming structures**.

Additional Key Words and Phrases: concurrent programming, retries, resource guarantees, scheduling, online policies

1 Introduction

An optimistic update loop prepares outside protection, validates its snapshot and publishes if still valid. Repeated failure can be bounded by recomputing under protection. This fallback limits calls but may protect all remaining work. A cached callback can instead reuse preparation on a match and recompute only on a mismatch. It reduces protected work, but a mismatch now pays for both preparation and recomputation. What is the least total work needed to guarantee a given bound on protected work?

We study a finite DAG of persistent jobs. Each executes a fixed-work opaque kernel using its own immutable input and the exact outputs of completed predecessors. The interface supplies cheap conditional publication, fresh protected completion and cached completion. A writer may change live inputs; a completed job stays completed. The resources are counted calls Q , all kernel work W , and its protected part L . These are operation-specific guarantees, excluding waiting and native-time predictions. We do not assume a measured latency SLA or equate shared-exclusion work with elapsed time.

The smallest dependency example already changes the design decision. Take works 8 and 16 and require two completing calls. With a supplied upper bound of one write, the order $8 \rightarrow 16$ admits worst $(W, L) = (32, 16)$. Reverse the dependency and the same protection cap requires $(40, 16)$. Requiring one policy to have the best possible protection guarantee at *every* hidden write bound also requires $(40, 16)$ in both orders. All-fresh minimizes total work at 24 but protects all 24 units. Figure 1 separates these contracts; the two coordinate maxima need not occur together.

Dependency	Supplied $B = 1, L \leq 16$		One optimal- L curve for all B	
	Least worst W	Worst L	Least worst W at 1	Worst L at 1
$8 \rightarrow 16$	32	16	40	16
$16 \rightarrow 8$	40	16	40	16
Independent	32	16	40	16

Fig. 1. Same works and $Q = 2$; different information/optimality contracts. All-fresh has $(W, L) = (24, 24)$ and does not meet the displayed protection cap. Values follow from the whole-kernel model, not timings.

We derive the complete independent-job frontier for every supplied budget and retry allowance, and extend it to every DAG without the cached flag when retries cover writes. At zero retry slack, we identify the price of one simultaneous optimal-protection curve, reduce the one-write DAG frontier to Lawler’s scheduling objective, and characterize hidden comparisons. A domination theorem makes immediate-preparation trees complete at every retry allowance. All lower bounds retain free inspections, old snapshots and charged retained preparations.

Fallback and runtime synchronization choice are established [5, 37, 38]. Scheduling with testing balances testing against execution costs [31, 34]; Pareto backward induction and budgeted robust games are inherited [8, 11]. Our claims concern exact joint resource values and the reduction from the specified completion-program class. Section 9 gives the explicit testing correspondence and distinguishes its objective and uncertainty budget.

Java executions test the resource and observation contract. Roslyn source modifications check selected preservation obligations and authored arrivals. Its mixed timings and stronger rebase operations outside the fixed-work model limit the practical interpretation: the resource formulas do not establish a general latency improvement.

2 Completion Interfaces and Resource Contracts

Write $\mathbb{N} = \{0, 1, \dots\}$. Jobs $J = \{1, \dots, n\}$, $n \geq 1$, form a finite DAG with predecessor set $\text{Pred}(i)$. Readiness requires all predecessors completed. Completion records an exact immutable output and persists despite later external writes. The objective is one successful foreground transformation per job, not a simultaneous snapshot or transaction. Policies are deterministic and causal; no randomized expected-cost guarantee is claimed.

Each ready job has a pure, opaque, whole kernel receiving its own-input identity and exact completed predecessor outputs. Every execution performs its fixed work $w_i > 0$, whether inside or outside protection. Partial/incremental evaluation, preparation before readiness and discharge by external work are excluded in this interface. A preparation is tied to its own-input identity and predecessor tuple. Captured outputs cannot be reconstructed by later reads of the mutable map.

States, histories and turns. Write a state as $(x, \mathcal{M}, \mathcal{R}, \ell)$: live own-input identities x , the partial map $\mathcal{M}$ of completed jobs to immutable outputs, a finite snapshot/record store $\mathcal{R}$, and program-local state ℓ . A record contains its job, captured identity, predecessor tuple and computed result. Records may be retained, copied and selected, but not fabricated or retagged without the corresponding computation. In Table 1, U means no selected record, C a selected record, possibly stale, and D permanent completion. Failed cheap calls clear the selection, leaving retained copies available. After every finite history, each unfinished job admits a legal own-key write with an identity never used for that job. Writes preserve completed outputs.

A finite visible history h lists program actions, returned values and completion events. A cached completion returns both its completed output and the Boolean result of the captured-identity comparison; that Boolean is visible to the policy. Completion publishes its saved output over the live own-key entry. The comparison flag is needed for the adaptive joint-work policies below; a pure kernel need not allocate a different result on recomputation. The program chooses a ready job and operation, or an unprotected inspection, preparation or local record operation. After that choice, the environment chooses a finite word of legal writes, then the operation executes. This environment move is the *gate*; it precedes comparison even within a callback. The mode/job is fixed before the gate; any later record selection uses only the prior store. Comparison and completing computation/publication are indivisible to writes; no guard survives return. Inspection honestly returns the live entry. Preparation computes a whole kernel from a captured immutable snapshot and completed parents; old captured snapshots are also allowed. Its pure body has no live read or

visible interaction. At completed-operation boundaries, contract that body with its returned record and full work; replay writes since capture in order before the next operation. Capture stays fixed. Section 7 gives the state relation, including pending writes: this does not place a completed output or its full cost at an unfinished concrete prefix.

The common start precedes any computed initialization: no computed record is given for free, and all whole-kernel preparation and computed initialization contributes to W . Old raw snapshots may be supplied, but computing their results is charged. Let W count all foreground kernel work, L its protected part, and Q validation, publication and completion calls, including failures. Preparations, initialization and external-writer calls are excluded from Q . These are separate resources. Common call overhead, waiting and nonkernel allocation are excluded. Neither a work counter nor a clock is implicitly visible to the policy. The supplement retains the earlier call-charge and partial-repair analyses as distinct objectives/interfaces. Let $\Omega_k(T)$ sum the largest $\min(k, |T|)$ works in T . Write $\Omega(T)$ for the total; omitted sets mean J . Set $\Omega_0 = 0$. These aggregate/allocation objects are inherited [9, 10].

Table 1. Whole-kernel operations. Every validation or callback adds one to Q ; preparation adds none. Entries show successor state and added (W, L) . Each callback holds protection only within that operation.

Operation	Start	Match	Mismatch
Prepare	U	C, $(w_i, 0)$	—
Cheap publication	C	D, $(0, 0)$	U, $(0, 0)$
Fresh callback	U	D, (w_i, w_i)	—
Cached callback	C	D, $(0, 0)$	D, (w_i, w_i)

A *cheap* call publishes on a match and returns only a success/failure flag. Conditional publication and a validation-only callback wrapped to discard its raw result have this same observation/resource transition (Section 7). The label concerns kernel work, not elapsed time. Two modes provide cheap publication and fresh completion; three also provide cached completion. Failed validation leaves the live entry unchanged. Inspection and local record operations cost no Q, W, L and never publish or retain a guard. Direct acquire/release, protection surviving return, cross-job callbacks and outside completion are absent from this entrypoint contract. These restrictions define the comparison; they do not assert completeness for arbitrary Java code or lock APIs. Each selected operation terminates. A play that never completes all jobs, including infinite free actions, has infinite objective; fairness of selected operations is an explicit assumption.

Strategies and admissibility. An informed strategy is a deterministic function $P(B, h)$; an unaware strategy is $P(h)$. Let $\mathcal{E}_b$ contain gate strategies using at most b writes along every play. For mode family j , $\mathcal{P}_j(B, r)$ consists of strategies completing with $Q \leq n + r$ against every $E \in \mathcal{E}_B$. By contrast, $\mathcal{U}_j(r)$ consists of one B -independent strategy satisfying that cap for every $b \geq 0$ and $E \in \mathcal{E}_b$. Every finite-write play against a fixed program is preserved by capping its environment at that play's write count. Worst cost at budget B is $\sup_{E \in \mathcal{E}_B} L(P, E)$; the two minima use different admissible classes.

LEMMA 1 (FRESH-WRITE AND FINITE-ADVERSARY FACTS). *In Section 2's game, a fresh own-key write at a selected i -operation's gate invalidates every retained i -record. Any concrete prefix has both a no-write and a fresh-write continuation whenever that extra write respects the environment's budget. An adversary bounded by b writes on every play forces any $P \in \mathcal{P}_j(b, r)$ to finish every job, regardless of free inspections or record retention.*

PROOF. Every retained i -record carries an identity previously captured from i ; the fresh write uses an identity outside that set. Copying or selecting records cannot change this. All choices before that gate, including inspection results and the selected job/mode, coincide in the two continuations. Only the next environment move differs, so no prior observation removes the fresh-write continuation. This does not erase the observation history or assert an observation-free normal form.

A globally b -bounded adversary belongs to $\mathcal{E}_b$. A play on which the program indefinitely inspects, prepares, or works on other jobs while leaving i unfinished violates admissibility on that very finite-write play. Consequently each job eventually completes. Its completing operation is either a fresh callback or a comparison using a prior record, exhaustively by Table 1. In the first case its whole kernel is protected without a write; in the second the fresh-write argument applies. No truncation of an unbounded adversary is used. $\square$

LEMMA 2 (WORK CHARGED BY USEFUL COMPARISON WRITES). *From the charged common start, consider a complete execution in which the adversary writes only fresh own-key identities at comparisons whose zero-write continuation would match. Let H be the multiset of resulting cheap failures and D the set of resulting cached mismatches. Then*

$$W \geq \Omega + \sum_{h \in H} w_{i(h)} + \sum_{i \in D} w_i.$$

Already-stale failures may consume calls, but need no extra charge in this inequality.

PROOF. At each selected write, charge the computation of the record that the zero-write continuation would select and match. That computation was paid before the gate, even if the fresh-write continuation actually selects an older record. The write invalidates every prior record for this job.

Before that job completes, the foreground cannot restore an earlier live identity: failed cheap calls, preparations and local actions do not publish. The restricted adversary never restores one either. A later useful comparison therefore requires a computed record for a different identity. Its charged computation is distinct from every earlier one for that job. Final completion needs one further distinct kernel: its protected execution or a matching preparation for the final identity. A useful cached mismatch has both the pre-gate charged preparation and this completing execution. Kernels for different jobs cannot share charges. Summing gives the bound even with early preparations, old snapshots, copied records and stale calls. This charges the entire execution; an arbitrary suffix need not owe a new $\Omega(S)$ because its records may be prepaid. $\square$

3 Universal Call and Protected-Work Guarantees

For unfinished jobs S , let $\mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$ denote the ready jobs.

THEOREM 1 (PROTECTED WORK UNDER A CALL CAP). *In Section 2's interface, fix integers $B, r \geq 0$. Among programs supplied with B that complete all jobs with $Q \leq n + r$ in every environment with at most B writes, the minimum worst protected work is*

$$L_3^*(B, r) = \Omega_{\min((B-r)_+, n)}, \quad L_2^*(B, r) = \begin{cases} 0, & B \leq r, \\ \Omega, & B > r, \end{cases}$$

with and without cached completion, respectively. Any DAG is permitted.

PROOF. At most r calls may fail to complete. Freshly prepared cheap attempts have disjoint read-to-comparison intervals; each failure needs a distinct write. If $B \leq r$ they give $Q \leq n + B$, $L = 0$. Otherwise switch after r failures to cached completion: at most $B - r$ writes remain, giving the stated bound. All-protected gives the two-mode upper bound.

For the two-mode lower bound when $B > r$, reject each cheap call with a fresh own-key write, stopping writes after the $(r + 1)$ st rejection. This uses at most $r + 1 \leq B$ writes. An admissible program cannot incur that many noncompleting calls, so all its jobs complete under protection.

For three modes, put $k = \min(B - r, n) > 0$ and fix the k heaviest jobs as targets. The adversary maintains f , the number of noncompleting target comparisons, and p , the number of completed targets. Initially both are zero. It writes only at a target's comparison gate, with a fresh identity, while $f \leq r$ and $p < k$. It stops all writes permanently when $f = r + 1$ or $p = k$. Counters follow actual outcomes: f increases on a noncompleting target comparison and p on every target completion. While writing is active, cheap target calls fail and cached ones recompute. Fresh completion protects w_i without a write. After stopping, calls follow their ordinary semantics.

Let v be the writes so far. At each program node, after the preceding operation returns, induction gives $v \leq f + p$: each write led to a failure or a cached completion; fresh completion only increases p . Immediately before any further write, $f \leq r$ and $p \leq k - 1$, giving $v + 1 \leq f + p + 1 \leq r + k \leq B$. Once stopped, no write is added. Thus this is unconditionally an $\mathcal{E}_B$ strategy, including plays that violate the call cap.

For an admissible program, $f = r + 1$ is impossible: completion requires n completing calls in addition to those failures, already exceeding $n + r$. Thus $f \leq r$ throughout its play. Lemma 1 forces every target eventually to complete; each such completion is fresh or is a cached mismatch, both protecting its whole work. Consequently $p = k$ and $L \geq \Omega_k$. Free inspections, stale selections and retained copies cannot evade either case. Predecessors still require their own completing calls, already included in the mandatory n . A nontarget predecessor changes neither f nor p and receives no write from this adversary. A target predecessor is counted exactly once in p . Thus dependencies delay targets without changing the write or call accounting. $\square$

THEOREM 2 (ONE POLICY FOR EVERY WRITER BUDGET). *Let $\mathcal{U}_j(r)$ contain the j -mode programs that receive no B and complete with $Q \leq n + r$ in every finite-write environment. Set $U_j(B, r) = \inf_{P \in \mathcal{U}_j(r)} \sup_{\# \text{writes} \leq B} L(P, E)$. Then*

$$U_3(B, r) = L_3^*(B, r), \quad U_2(B, r) = \Omega \mathbf{1}[B \geq r].$$

One policy per mode family attains these values simultaneously for all B and positive works on any DAG, using only readiness and a global cheap-failure counter.

PROOF. Switch after r cheap failures to a fresh read/preparation with cached completion (three) or to fresh protected completion (two). This uses at most r extra calls universally. At $B < r$ no protection is needed; at $B = r$ cached completions after switching still match. Theorem 1 supplies the remaining upper and three-mode lower bounds.

For two modes when $B \geq r$, reject every cheap call with a fresh own-key write until r rejections, then stop. Only protected calls complete before this. Afterwards, another rejectable call could fail in a different finite environment, violating the call cap. Thus all jobs complete under protection with at most $r \leq B$ writes. That additional hypothetical write is absent from the lower-bound execution. Retained preparations and inspections cannot prevent a fresh write before comparison. $\square$

For equal works and $B > r$, the ratio is $n/\min(B - r, n)$. At $B = r$, only an informed two-mode program knows no write remains and may risk another cheap call. The unaware class must also be safe in extensions with another write, causing the boundary shift. A deployment that supplies a certified bound and requires the cap only within it instead uses $\mathcal{P}_j(B, r)$; that is a different information contract.

Readiness constrains when a target is attempted, but persistence requires every target eventually. Thus DAG edges do not change this frontier. They become consequential when total work is constrained (Section 5).

Per-update and shared-batch call allowances are different contracts. The supplement preserves the full combined local/shared formulas and proofs. Source comparisons below report their distinct caps; a lower observed protection count under a looser call cap is not a same-contract improvement.

4 The Total Work Needed for Optimal Protection

The protected-work frontier alone permits arbitrarily wasteful preparation. We now optimize total work W under a bound on L . First consider $r = 0$, when every counted call must complete one job; Section 4.2 then permits arbitrary r . The common start precedes all computed initialization; every whole-kernel preparation, including discarded or retained records, contributes to W . We study the pair of separate suprema $(\sup_E W, \sup_E L)$. Their maximizing executions need not coincide, so this pair does not determine $\sup_E (W + \lambda L)$.

4.1 One Policy with an Optimal Curve at Every Budget

Let $\mathcal{V}$ contain the policies in $\mathcal{U}_3(0)$ that also satisfy $L(P, E) \leq \Omega_b$ for every $b \geq 0$ and $E \in \mathcal{E}_b$. Theorem 2 makes this the simultaneous optimal- L contract, rather than just universal completion.

THEOREM 3 (TOTAL WORK UNDER SIMULTANEOUS OPTIMAL PROTECTION). *For every DAG and $B \geq 0$,*

$$\inf_{P \in \mathcal{V}} \sup_{E \in \mathcal{E}_B} W(P, E) = \Omega + \Omega_B.$$

One policy attains this whole curve: choose any ready job, prepare immediately before its cached completion, and always complete with that operation.

PROOF. For the upper bound, every job executes once outside protection and executes once more iff its cached completion mismatches. Each such mismatch requires an own-key write in its distinct preparation-to-comparison interval. At most B distinct jobs duplicate, yielding $W \leq \Omega + \Omega_B$ and $L \leq \Omega_B$, with $Q = n$ in every finite-write environment.

For the lower bound, target $k = \min(B, n)$ heaviest jobs. Write a fresh identity at each target comparison, with an unconditional cap of k writes, and nowhere else. A cheap failure is inadmissible under $Q = n$. Every target completes fresh or through cached mismatch; no target comparison can match before the write cap. If $j \geq 1$ targets complete fresh, at most $k - j$ target comparisons and writes occur, yet all k targets are protected. This gives $L \geq \Omega_k > \Omega_{k-j}$, contradicting $P \in \mathcal{V}$. Thus all k targets duplicate their whole kernels. Every other job still needs one charged kernel, giving $W \geq \Omega + \Omega_k$. For $k = 0$ use the mandatory Ω bound. Lemma 1 covers progress, inspections and retained records; dependencies only delay targets. $\square$

This theorem does not say every optimal- L policy has the displayed W curve: extra preparations can waste work. It identifies the least total work possible within that simultaneous contract and an attaining policy.

4.2 The Complete Frontier for Independent Jobs

A supplied write bound permits a different policy for each resource contract. Here allow arbitrary retry slack $r \geq 0$. Sort independent jobs by positive work $a_1 \leq \dots \leq a_n$ and let $A_k = \sum_{j=1}^k a_j$, $A_0 = 0$. Put $h = \min(r, B)$ and $p = B - h = (B - r)_+$.

THEOREM 4 (INDEPENDENT-JOB FRONTIER). *For independent jobs, the exact $(\sup W, \sup L)$ frontier over $\mathcal{P}_3(B, r)$ is the nondominated subset of (Ω, Ω) and*

$$(\Omega + C_k, \Omega - A_k), \quad 1 \leq k \leq n - p,$$

$$C_k = ha_k + A_{k+p-1} - A_{k-1}.$$

For $p = 0$ the prefix difference is zero. If $p \geq n$ only (Ω, Ω) remains; at $B = 0$ the frontier is $(\Omega, 0)$. Sorting and a prefix-sum sweep construct the frontier in $O(n \log n)$ arithmetic/comparison operations.

PROOF. For selected k , process jobs in ascending work order. Count matching completions s and cheap failures f . Before each preparation, if $s = k$, finish the suffix fresh; otherwise prepare/cheap when $f < r$ and prepare/cached when $f = r$. Stop when all jobs finish. This policy reads no B after k is selected.

Let m count cached mismatches. Disjoint invalidation intervals give $f + m \leq B$; cached mode requires $f = r$, so $m \leq p$. If all jobs finished before $s = k$, then $n = s + m \leq k - 1 + p < n$, a contradiction. Hence exactly k matching jobs precede the fresh suffix. Their weight is at least A_k , giving $L \leq \Omega - A_k$, attained without writes.

Cheap failures occur before cached mode, at rank $s + 1 \leq k$, and cost at most ha_k . The j th cached mismatch has rank $s + j \leq k + j - 1$. Thus $W - \Omega \leq ha_k + \sum_{j=k}^{k+p-1} a_j = C_k$. To attain it, match the first $k - 1$ jobs, fail cheap h times on job k , then cause p cached mismatches at ranks $k, \dots, k + p - 1$. The next job $k + p \leq n$ matches and triggers fresh completion. For $p = 0$, job k itself matches after the failures. The call bound is $Q \leq n + h$ within budget and $Q \leq n + r$ against any finite number of writes.

For the lower bound, target ranks $k, \dots, n$. A globally B -bounded adversary writes only at target comparisons whose zero-write continuation would match. Already-stale cheap calls receive no write; stale cached calls protect without one. If the budget is not exhausted, every target completes protectively, giving $L \geq \Omega - A_{k-1}$. Otherwise let $f \leq h$ writes cause cheap failures; the other $B - f$ cause cached mismatches at distinct targets. Lemma 2 gives

$$W - \Omega \geq fa_k + \sum_{j=k}^{k+B-f-1} a_j \geq C_k.$$

The actual distinct targets ensure this window exists. Increasing f replaces its last weight by a_k and cannot increase it; at $f = h$ it equals C_k . Wasted stale failures consume retry allowance without reducing this bound.

Consequently worst $L < \Omega - A_{k-1}$ forces worst $W \geq \Omega + C_k$. Since C_k is nondecreasing, a cap $M < C_1$ forces all work protected. Otherwise choose the largest k with $C_k \leq M$; the next index's target bound forces $L \geq \Omega - A_k$. At $k = n - p$, Theorem 1 supplies the same lower bound Ω_p . The attaining policies complete the frontier; nondominance removes tied thresholds. At $B = 0$, select $k = n$; at $p \geq n$, mandatory work and the protected-work lower bound give all-fresh. $\square$

One fixed r, k policy attains its stated pair for every $B \leq r + n - k$. At the next budget, matching $k - 1$ jobs, failing cheap r times and mismatching every remaining job gives $L = \Omega - A_{k-1} > \Omega - A_k$. Thus the constant protection guarantee's range is sharp. Selecting k still uses the supplied contract; this is not one optimal- L curve for all unknown budgets.

At $r = 0$, the optimal- L corner is

$$\min_{\substack{P \in \mathcal{P}_3(B, 0) \\ \sup L \leq \Omega_B}} \sup W = \begin{cases} \Omega + \Omega_{B+1} - w_{\max}, & B < n, \\ \Omega, & B \geq n. \end{cases}$$

Compared with Theorem 3, the extra total work required by the *simultaneous optimal-L contract* is $w_{\max} - w_{(B+1)}$ for $B < n$, using nonincreasing works, and Ω for $B \geq n$. This compares two specified policy classes, not unrestricted pointwise ignorance. Ties and $B = 0$ can eliminate the first gap.

4.3 When Retry Slack Covers the Write Budget

PROPOSITION 5 (AN ORDER-INDEPENDENT FRONTIER WITHOUT THE CACHED FLAG). *If $r \geq B > 0$, on every DAG the least worst L under $W \leq \Omega + M$ is*

$$\sum_{i: Bw_i > M} w_i.$$

Its attaining policy needs only cheap success/failure, even under the output-only cached contract of Section 6.2.

PROOF. Choose the k smallest jobs for cheap calls and all others fresh; follow any topological order until B cheap failures, then prepare/cache every unfinished job. Duplicate work is at most Ba_k , and protected work at most $\Omega - A_k$. Repeated writes at a maximum chosen job and the no-write path attain the separate bounds. Theorem 4's target lower bound uses no execution order and specializes to $C_k = Ba_k$. Tied thresholds give the displayed cap form. $\square$

For the 8/16 example, one spare call and supplied $B = 1$ permit $(W, L) = (32, 16)$ and $(40, 0)$ in either edge direction. Enough failures can reveal budget exhaustion through cheap results; the cached comparison need not be exposed. This does not eliminate the observation issue when $r < B$ or when one optimal- L curve is required at every unknown budget.

5 Precedence Determines the One-Write Frontier

DAG edges did not affect the protected-work value. They affect the least total work that realizes it. Consider a supplied $B = 1$ and a cap $W \leq \Omega + M$, $M \geq 0$. Define a scheduling instance by

$$(p_i, \tau_i) = \begin{cases} (w_i, 0), & w_i > M, \\ (0, w_i), & w_i \leq M. \end{cases}$$

Call the first jobs heavy and the others light. For a topological order π , $C_i(\pi)$ is the sum of p_j through job i .

THEOREM 6 (REDUCTION TO PRECEDENCE-CONSTRAINED SCHEDULING). *For arbitrary DAGs, $B = 1$ and $Q \leq n$, the minimum worst protected work subject to worst total work at most $\Omega + M$ is*

$$\ell(M) = \min_{\pi \text{ topological}} \max_i \{C_i(\pi) + \tau_i\}.$$

Lawler's backward minimum-cost sink rule [30] computes the order. Sweeping $M \in \{0, w_1, \dots, w_n\}$ and removing dominated pairs gives the entire W/L frontier.

PROOF. For an order π , complete heavy jobs fresh and light jobs by immediate prepare/cache until a mismatch. After that mismatch, prepare/cache the suffix. At most one job duplicates and its weight is at most M , so $W \leq \Omega + M$. A no-mismatch execution protects all heavy work. A mismatch at light i protects its earlier heavy prefix plus w_i , and nothing later. These cases give $\max_i (C_i + \tau_i)$.

For the lower bound, follow any admissible program's no-write execution. Every completing call is fresh or cached: a cheap call has a one-write failing continuation. Any matching cached call on i admits an own-gate fresh write that forces $W \geq \Omega + w_i$. Therefore every heavy job must be fresh. A cached call already stale without a write can be classified as fresh in this argument: it already protects w_i , and its wasted preparation only strengthens the work lower bound.

The no-write path forces at least its total fresh work in L . Each matching cached job i also gives a one-write witness with protected work at least its preceding fresh prefix plus w_i . Convert an optional fresh light job to cached in this lower-bound expression: later fresh prefixes decrease, and its new own term is at most the old total fresh work. Repeating cannot increase the expression. The remaining fresh jobs are exactly the heavy ones, producing (p_i, τ_i) and the claimed minimum over topological orders. This proof covers arbitrary preparations and subsequent value-dependent choices; it does not assume the normal form of Section 6.

For completeness, the inherited scheduling exchange is short. Among sinks of the remaining graph, select j with minimum τ_j and place it last. In an optimal order with total remaining processing T and last job k , moving sink j to the end weakly decreases intervening completion times. Its new term $T + \tau_j$ is no larger than the old last term $T + \tau_k$, since k was also a sink. Repeat on the remaining graph. This works with zero processing times. Changing M matters only when a weight crosses the threshold; that job's own term is unchanged and later processing prefixes decrease. Hence $\ell(M)$ is nonincreasing, and the finite sweep is complete. $\square$

This is an equivalence from the completion-program resource constraint to a classical scheduling objective, not a new scheduling algorithm. A straightforward sink implementation takes $O(n^2 + |E|)$ per threshold and $O(n^3 + n|E|)$ for the sweep.

PROPOSITION 7 (WHEN THE OPTIMAL PROTECTION CAP PERMITS A SAVING). *At $B = 1$ and $L \leq w_{\max}$, the minimum worst total work is $\Omega + w_{(2)}$ if the maximum is unique and its job is a sink; otherwise it is $\Omega + w_{\max}$. Set $w_{(2)} = 0$ for one job.*

PROOF. If $M < w_{(2)}$, at least two heavy jobs have total exceeding $w_{\max}$. If the unique maximum has a successor, that successor's term exceeds $w_{\max}$ whenever $M < w_{\max}$. Tied maxima also force excessive heavy total below this threshold. Conversely, for a unique maximum sink, $M = w_{(2)}$ and placing that job last gives objective $w_{\max}$. At $M = w_{\max}$ all-cached always attains it. $\square$

For $8 \rightarrow 16$, the informed policy first prepares/caches the 8-unit job. On match it completes the 16-unit job fresh, giving $(W, L) = (24, 16)$ without writes. On mismatch it prepares/caches the large job, giving $(32, 8)$. Thus the pair of worst values is $(32, 16)$, compared with $(40, 16)$ under the simultaneous contract. Reversing the edge requires $(40, 16)$ even with the supplied budget. Neither result predicts elapsed time.

6 A Complete Finite Policy Class for Arbitrary Retry Slack

The preceding polynomial regions have direct lower bounds. For other DAG/budget combinations we justify finite policy search against the full class of programs with free inspections, early preparation after readiness, and retained records.

THEOREM 8 (RESOURCE DOMINATION BY IMMEDIATE-PREPARATION TREES). *For every $P \in \mathcal{P}_3(B, r)$ there is a finite fresh/cheap/cached tree T such that*

$$\forall E \in \mathcal{E}_B \exists E' \in \mathcal{E}_B : (Q, W, L)(T, E) \preceq (Q, W, L)(P, E').$$

Its depth is at most $n + r$, and it prepares only immediately before its selected cheap/cached node.

PROOF. Restrict the adversary to no writes except an optional fresh own-key write at a comparison whose zero-write continuation matches. Fix a concrete fresh identity at each branch history. A fresh call gives a unary fresh node. A matching cheap call gives completing and fresh-write-failing children; a matching cached call gives two completing children. An already-stale cached call becomes unary fresh, retaining its original zero-write continuation. Delete an already-stale cheap call and retain its failing continuation and fixed local choices in the witness history.

If the original prefix has u completions, h useful cheap failures and s deleted stale failures, its calls number $u + h + s$; the tree has $u + h$. Its unused failure allowance may increase, but need not be spent. All represented paths have at most $n + r$ nodes. Between them, no-more-write computation and deleted calls must terminate, or the original program would not complete in that finite-write environment. Finite branching and bounded depth therefore yield a finite tree.

Cap the restricted adversary at B and replace a zero-remaining-budget suffix by ready prepare/cached completions. Useful comparisons cost counted calls; the original cap forces completion along every represented path.

On a complete path let H be its binary cheap failures and D its binary cached-mismatch jobs. Lemma 2 lower-bounds original work by $\Omega + \sum_H w_i + \sum_D w_i$. Original protected work is at least the sum of the extracted fresh-node works and $\sum_D w_i$; a collapsed stale cached call already protects its kernel. A known-zero-budget suffix substitution can only lower protected work. Deleted stale cheap calls need no separately assigned preparation charge.

Realize the fixed tree using actual current inputs and exact saved parents, ignoring their values for scheduling. Prepare only at its selected cheap/cached node, never speculatively for a fresh node. Readiness follows the same completed set. Actual failures/mismatches have disjoint preparation-to-comparison intervals and inject into distinct actual writes; irrelevant or identity-reusing writes may remain invisible. Thus a path with $d = |H| + |D|$ bad comparisons requires at least d writes. Exactly d own-gate fresh writes realize that path and its original concrete witness. Define the witness adversary with an unconditional cap d , stopping after that many writes on every history. It is globally d -bounded, rather than merely spending d on one play.

The tree's exact path costs are $Q = n + |H|$, $W = \Omega + \sum_H w_i + \sum_D w_i$, and $L = \sum_{\text{fresh}} w_i + \sum_D w_i$. They are no greater than the original witness's. Any actual tree execution with at most B writes is therefore matched to an original environment in $\mathcal{E}_d \subseteq \mathcal{E}_B$. Correct outputs follow from the operations' semantics, not from reproducing the original program's concrete values under the same environment. $\square$

6.1 Exact Pareto Recurrence

Let $\mathcal{F}(S, b, a)$ contain nondominated suffix-tree bounds (e, l) , where S is unfinished, b is a conservative remaining write allowance, a remaining noncompletions, $e = \max W - \Omega(S)$ and $l = \max L$. Set

$$\mathcal{F}(\emptyset, b, a) = \mathcal{F}(S, 0, a) = \{(0, 0)\}.$$

For ready i , write $R = S \setminus \{i\}$ and take the nondominated union of

$$\begin{aligned} \text{fresh:} & \quad (e, w_i + l), \quad (e, l) \in \mathcal{F}(R, b, a); \\ \text{cached:} & \quad (\max\{e_0, w_i + e_1\}, \max\{l_0, w_i + l_1\}); \\ \text{cheap, } a > 0 : & \quad (\max\{e_0, w_i + e_1\}, \max\{l_0, l_1\}). \end{aligned}$$

Both comparisons choose their matching child in $\mathcal{F}(R, b, a)$. Cached chooses its other child in $\mathcal{F}(R, b - 1, a)$; cheap chooses it in $\mathcal{F}(S, b - 1, a - 1)$. Children are selected independently.

A cheap failure leaves its job unfinished, so its lost preparation adds an extra w_i but no protected work. Cached mismatch also duplicates a kernel. Separate maxima may occur on different paths; the pair does not determine $\sup(W + \lambda L)$. Future readiness depends on S and tree costs on works, modes and outcomes. The table indexes attainable suffix bounds, not equivalent original histories; emitted policies retain their selected witness nodes. Monotone maxima justify dominated-child removal. This proves the recurrence for trees, and Theorem 8 proves completeness for arbitrary programs. Pareto backward induction is inherited.

There are at most $(r + 1)(\min(B, n + r) + 1)2^n$ states, with possibly exponential frontier sizes. The independent-job formula, enough-retry DAG formula and zero-retry one-write Lawler reduction identify polynomial regions; a general fixed-order theorem is not claimed.

The supplement also extracts one budget-independent tree for a universal program by retaining all branches up to depth $n + r$. Finite curve enumeration and an early-coordinate lower bound do not establish existence of a least total-work curve at positive retry slack. Theorem 3 establishes the full simultaneous result only at $r = 0$.

6.2 The Frontier When the Comparison Is Hidden

Consider a weaker observation contract. Each pure kernel performs its prescribed work but returns the same immutable per-job object z_i on every invocation. Completion publishes z_i over the compared live entry. The cached primitive's *entire visible transition* exposes only that output and completion: it does not retain a compared current-input reference, Boolean, comparison-dependent record selection or callback-local side effect. Kernel internals, work counters, allocation IDs and clocks remain hidden. Free outside inspections and immutable old snapshots are still allowed. This condition is stronger than merely returning a constant: a callback that saves the current input elsewhere adds an observation.

THEOREM 9 (OUTPUT-ONLY RESOURCE FRONTIER). *Under this observation contract, the $r = 0$ frontier at a supplied B on any DAG is*

$$\text{ND}\{(\Omega + \Omega_B(C), \Omega(J \setminus C) + \Omega_B(C)) : C \subseteq J\}.$$

A fixed choice of cached jobs C and fresh jobs $J \setminus C$, executed in any topological order with immediate preparation, attains each candidate. At $L \leq \Omega_B$, least worst W is $\Omega + \Omega_B$ for $B < n$ and Ω for $B \geq n$.

PROOF. For $B \geq 1$, follow an arbitrary program's no-write execution. Cheap is inadmissible because one gate write would cause a failure. Let F contain fresh completions and cached calls already stale without a write; let C contain matching cached calls. A stale call already protects its whole kernel.

For any $H \subseteq C$, $|H| \leq B$, write once at each target's comparison and nowhere else. Before each target call, all outside-visible state agrees with the no-write run. The fresh input invalidates all retained records, but completion overwrites it with z_i and restores the same complete visible state. Induction gives identical inspections, retained-snapshot comparisons, local choices and later modes throughout this restricted family. It does not erase data dependence in arbitrary environments.

Global charging gives $W \geq \Omega + \sum_{i \in H} w_i$ and $L \geq \Omega(F) + \sum_{i \in H} w_i$ on the same execution. Choosing the heaviest cached targets gives the partition pair. The fixed partition policy attains it by disjoint preparation-to-comparison intervals; target writes attain both maxima. This proves domination and exactness. If $B < n$ and F is nonempty, $\Omega(F) + \Omega_B(C) > \Omega_B(J)$: it either equals Ω or includes the B heaviest works and an additional positive contribution. Thus the optimal- L cap requires the all-cached partition. At $B = 0$ prepare/cache attains $(\Omega, 0)$; at $B \geq n$ all-fresh attains (Ω, Ω) . $\square$

For $1 \leq B < n$, all-fresh and the candidates caching the k lightest jobs suffice:

$$(\Omega + A_k - A_{k-B}, \Omega - A_{k-B}), \quad k = B + 1, \dots, n.$$

To see this, list any cached set's sorted indices as $c_1 < \dots < c_m$. If $m \leq B$, all-fresh dominates. Otherwise set $t = c_{m-B+1}$ and $k = t + B - 1 \leq n$. The prefix through k has top- B sum $\sum_{j=0}^{B-1} a_{t+j}$ no greater than the old one, while its protection saving A_{t-1} includes all old $m - B$ smaller cached jobs. It improves both coordinates. Sorting/filtering takes $O(n \log n)$ plus a topological order. It chooses the cached set; DAG edges do not change values.

For independent $(1, 2)$ and $B = 1$, a visible comparison permits $(\max W, \max L) = (4, 2)$, but the output-only cap requires $(5, 2)$. More generally the Boolean reduces the optimal- L corner by $w_{\max} - w_{(B+1)}$ for $B < n$, and by zero for $B \geq n$. Ties and $B = 0$ can eliminate the saving. In the one-write DAG example, hiding the comparison makes both edge directions require $(40, 16)$. The native controls below instantiate the missing observation. Fresh output allocation with identical-reference return on match is another sufficient observation contract; purity alone supplies neither.

7 Actual External Writes in Java

Use Java 17's `ConcurrentHashMap` with one foreground controller, permanent distinct keys, deeply immutable non-null values and never-reused own-key write identities. Values have identity equality. Kernels and external writers preserve each job's tag and fixed shape; kernels use only immutable own snapshots and completed parents. Key hash/equality/comparison methods are stable. Kernels and key methods terminate normally without side effects or recursive map updates. Replacement can test equals, so identity equality matters [15, 16]. We assume normal progress of selected API calls, not progress from finite writing alone.

The wrapper uses `replace(key, expected, result)` or validation-only compute for cheap publication. The latter returns the prepared result on match, otherwise the unchanged entry, but exposes only its match flag. Unwrapped mismatch results differ: `replace` returns false and `compute` returns the current value. Cached callbacks recompute on mismatch and return the callback-local identity comparison Boolean alongside their saved output; fresh callbacks always compute. The relation preserves opaque-reference equality, payloads, wrapper results, completed parents and selector state, and separately the resource counts Q, W, L . Work counters, clocks, numeric allocation IDs, logs and scheduling probes are excluded from policy observations; the native selector/payload projection erases its diagnostics. Reference renaming preserves aliasing and permits fresh output allocation.

State and operation correspondence. Let H be the heap. An injective reference map η covers the live map, pending writes, saved results and available handles; extensions never change existing correspondences. In $(x, \mathcal{M}, \mathcal{R}, \ell)$, set $\mathcal{M}(i) = \eta(\text{done}[i])$ permanently from the saved completing output. The selectable store $\mathcal{R}$ contains exactly available snapshots and computed handles $(i, \eta(s), \mathcal{M}|_{\text{Pred}(i)}, \eta(z))$; discarded ghost-history results are not selectable. The native loop keeps only current handles; the larger strategy class requires actual retention. Locals/selections map to ℓ .

For an ordered queue Δ of already committed external assignments deferred in normalization, the boundary relation is

$$\text{decode}_{\eta}(H.\text{map}) = \text{apply}(\Delta, x), \quad b_H = b_x + |\Delta|.$$

The b 's count committed publications, not a diagnostic counter incremented inside a writer callback before its store. Decoding follows forwarding nodes at stable boundaries; Δ replays assignments, not callbacks.

The API specifies atomic compute with one callback invocation [14]. In the pinned source [16], the monitor/head check is at lines 1931–1932; list/tree remapping and stores are at 1940–1942/1975–1978. Snapshot comparison occurs in our callback; same-key writes cannot intervene before its store. `replaceNode` saves the old value (1131–1136, 1155–1160); conditional replace converts a non-null old value to success (1563). `compute` returns its saved Value (2003). Protection ends before return, allowing a newer map value. Successful cheap publication records the saved prepared output; fresh/cached completion records the returned Value. Neither rereads the map. Captures/inspections use the per-key retrieval guarantee and forwarding lookup (934–955).

Consequently a completed preparation adds $(Q, W, L) = (0, w_i, 0)$ and its record; a cheap call adds $(1, 0, 0)$ and completes exactly on match. Fresh completion adds $(1, w_i, w_i)$; cached match/mismatch adds $(1, 0, 0)/(1, w_i, w_i)$. Every completion stores its saved output in $\mathcal{M}$. A failed cheap call clears selection while retaining available copies. These exhaust Table 1's actions/results.

THEOREM 10 (RESOURCE BOUNDS FOR ACTUAL WRITES). *Under these observation, wrapper, API and progress assumptions, Theorems 1–2's call/protected-work bounds and Sections 4–6's joint-work results hold for actual external writes under each result's specified observation contract, with separate or shared bins. Completed-operation traces preserve the listed results and Q, W, L ; this is not a mechanically verified JDK/JMM theorem.*

PROOF. Group each completed pure kernel with its actual operands, result and full work. Writes since capture enter Δ and flush before the next foreground operation, including an inspection. The output exists at this completed boundary; unfinished kernel instructions are not zero-cost stutters. Writes after publication but before return also enter Δ : the result is fixed, and $\mathcal{M}$ uses it while the relation permits a newer live value. A terminal write suffix flushes any remaining queue, preserving the final map. No write is added. Pure structure changes copy value references (transfer 2505–2524, treeification 2675–2677, untreeification 2697); concurrent publications remain write transitions. The API/source properties connect stable boundaries; node copying alone is not a JMM proof.

For the explicit upper-bound policies, fresh read-to-comparison intervals are disjoint, so each mismatch needs a distinct committed write. Bounded finite wrapper segments terminate under the progress assumption, transferring the call/work upper bounds. This does not infer arbitrary low-level termination from completed traces.

For lower bounds, suspend the controller after its job/mode and finite prior store are fixed but before API entry. A normally completing own-key writer installs a fresh legal reference, then the selected call runs. No foreground monitor is held, even with bin collisions; every prior target snapshot is invalidated. Cheap calls fail, cached calls recompute, and fresh calls already protect their whole kernel. Theorem 1's counters still give $v + 1 \leq f + p + 1 \leq r + k \leq B$ before each target write. Its bounded adversary forces completion despite free actions. Theorem 2's extra write witnesses admissibility in a separate finite environment, not the evaluation trace. Completed parents and the one-job-per-call rule preserve these arguments. The same pre-entry fresh writes realize the joint-work target adversaries. Charging all preparations from the common start transfers their global W bounds. The callback's local Boolean implements each extracted tree branch, even if recomputation returns the same canonical result; no diagnostic work counter is used by the selector. $\square$

Operational accounting. Earlier kernels preserve array length $8w_i - |\text{Pred}(i)| > 0$ and execute $8w_i$ counted iterations, so $W = \text{work}/8$; L counts protected iterations and Q callbacks/conditionals. New joint-work wrappers use exactly w_i loop units and include canonical outputs. The zero-retry wrapper exposes the cached comparison; the enough-retry wrapper needs only cheap flags, exercises both cheap operations and verifies an actual nine-node tree bin. Earlier $n \leq 5$ fixtures bypass these cases. Counters/structure inspection are checker-only; waiting, allocation, traversal and writers are outside W, L . Resize during execution remains outside the finite matrix.

8 Formal Corroboration and Operational Evaluation

RQ1 checks the joint resource formulas and observation contract; RQ2 checks source preservation and costs. Inputs precede execution; arrival reanalysis is post-outcome. All failures, timeouts, invalids and corrections remain archived.

An Apple M2/24 GiB runs Python 3.10.9/3.12.14 and OpenJDK 17.0.19; inspected map source matches the cited version but its binary was not rebuilt. Hashes bind inputs, code, commands and traces. Codex assisted design, checkers, source comparison, proofs, analysis and writing; Claude supplied separate proof/direction critiques and manuscript reviews. Measurements are archived executions. These are author-side checks, not mechanical proofs, independent third-party reproduction or human studies.

An *input* fixes graph, works and prices/footprints; a *root* adds budget/retries. Native maxima over finite replay groups need not coexist. Table 2 separates overlapping studies.

8.1 RQ1: Bounded Corroboration of the Proofs

Table 2. Bounded checks of the retained claims. The studies overlap in their source inputs; their denominators are reported separately. Finite agreement is not an all-program proof.

Obligation	Evidence and comparison
General call/ L values	Direct minimax versus formulas: 265,629 agreeing roots from 5,421 DAG/weight inputs.
General retry frontier	Immediate and retained-preparation finite models: 96,795 agreeing cases; $n \leq 4$, works $\{1, 2, 4\}$, $r = 0, 1, 2$.
Polynomial formulas	Retrospective: 1,998 independent and 32,526 enough-retry DAG cases (720 overlap). Prospective: 13,850 agreeing roots and 540,230 independently interpreted paths.
Unknown-budget curves	713 roots including two longer-chain counterexample fixtures: the early optimal- W prefix agrees; later least-curve existence is unproved.
Zero-retry regressions	26,844 Pareto cases; 18,390 scheduling cases; 2,004 independent corners; 237 simultaneous curves; 2,367 output-only cases. All agree with their stated comparators.
Native policies	2,076 zero-retry executions; 12,224 one-retry executions with two cheap wrappers and verified tree bins. All meet outputs and resource caps.
Flag-ignoring controls	Four of 52 violate L as predicted; all other 48 are retained.

The zero-retry recurrence retains every nondominated pair; all topological orders and a separate operation-path interpreter check its one-write values. Its 26,844 frontiers also agree with the union over fixed topological orders, which does not prove a general fixed-order theorem. The output-only study enumerates every partition and allowed bad subset for $n \leq 5$, works $\{1, 2, 4\}$ and $0 \leq B \leq n+1$; 950 of 2,367 cases have positive Boolean value at the optimal- L corner.

For arbitrary retry slack, the prospective independent-job study fixes 461 sorted multisets, $n \leq 6$, works $\{1, 2, 4, 7, 11\}$, $r = 0, \dots, 3$ and $B = 0, \dots, n+r$. The closed frontier, all-ready-job oracle and two separately fixed threshold-policy variants agree on the same 13,850 inputs, each with 540,230 interpreted paths. The simplified policy also violates its constant L cap at the first excluded budget in all 9,240 boundary cases; $Q \leq n+r$ still holds. All paths and witnesses remain. The 96,795-case study additionally allows retained preparations in its finite model. Neither finite class proves the unrestricted-program reduction; that obligation is discharged by Theorem 8's global charging argument.

The zero-retry Java matrix uses six fixtures: one job, both orders and the independent pair of 8/16, a fork and tied works. It crosses work thresholds, separate/shared bins, allocating/canonical kernels, and no/single writes by position, target and phase (before preparation, before call, or after return before saving). An actual writer commits each write; the checker reconstructs invocation

costs, identities and exact saved parents. Four of 52 flag-ignoring controls give $L = 24 > 16$; 70 mismatches return the identical prepared result reference. In 864 executions a saved milestone differs from its final live entry, yet successors use the saved output.

A separate supplied- $B = 1, r = 1$ matrix adds a nine-job chain, verifies all keys remain present, and checks one actual nine-node tree bin in each of 4,336 colliding executions. It crosses both cheap wrappers with both kernel-output contracts; all 6,112 wrapper pairs agree on payloads, traces and resources. Across all 12,224 executions, 304 cheap calls fail and 1,056 cached calls run. All 176 threshold groups attain their specified worst W, L, Q ; 1,848 schedules whose late write point is not reached remain explicitly unissued. The selector observes only cheap success/failure. All writer executors terminate. Compilation plus execution took 2.214 seconds; this is accounting verification, not a timing benefit.

Earlier aware/unaware records (9,314/17,714), six scaling timeouts, charged/partial-repair studies and counterexamples to simple positive-retry W rules remain. The new frontier does not erase them. Earlier native cost fitting succeeds in only two of eight cases.

A controlled operational consequence. An earlier Java probe compares two/three-mode protection for dependent works (1, 2, 3, 5)s, $r = 0, 1, 2$ and $B = r$ initial failures. Both have $Q = 4 + r$, $W = (11+r)$ s; protection is 11s/0. Across 4,320 measurements and 1,440 warmups, forced shared-bin waiting at the largest scale falls from 26.8 ms to 31.2 μ s. All 24 cells remain, including submicrosecond or reversed disjoint-bin differences. The readiness hook creates blocking; this establishes no natural-arrival or latency bound.

8.2 RQ2: What Does the Source Integration Preserve?

Integration and limits. Roslyn's Workspace.cs loop at 6c4a46a3 transforms an immutable solution outside a semaphore, validates identity inside and retries [6, 7]. We modify only SourceText: after r shared failures, two modes transform inside; three reuse preparation on match or recompute inside. The selector receives no writer tables. Clearing scratch state preserves replayability. Our same-text reentry deadlock required restoring the pre-lock no-op; four missing-document timeouts required releasing protection before replaying the captured transformation. All 14 fixed error cases then complete; these defects belong to our patches.

Lazy work, allocation and VersionStamp generation limit transfer to counts. Custom loaders, changed-text reentry and arbitrary compiler behavior remain outside scope; no upstream numerical cap or SLA was found. Prepared rebase uses WithDocumentContentsFrom when solution/related-document IDs and affected project states match, otherwise three-mode fallback. Even reusable contents need a protected merge; its guard, merge and invalidation costs have no calibrated additive law.

Source projection and semantic checks. Pinned net8 Roslyn assemblies run on a pinned .NET 10 RC runtime. Four projects give 3,837 source files, 4,019 document occurrences and 40.8 MB; two package-external items were excluded before outcomes. Updates append comments. The grid covers caps, concentrated/distributed writers, linked targets, no-ops, equal-content distinct texts and absent/deleted targets. All 236 cases agree on 2,190 events, 1,253 publications and 2,653 snapshots. The checker binds payloads, publication chains and 4,016 untouched identities/versions/metadata; 12 corruptions reject.

Rebase passes 18 error-reentry and five same-text controls; patch04 passes 14 and four. The original deadlock and four timeouts remain. A 96-process study varies source, parse options, references and linked text across eight arms and cold/warm caches. All 2,556 project projections match declared source, symbol, diagnostic and emission expectations; eight corruptions reject. Workspace/direct compilations share Roslyn and cover authored programs.

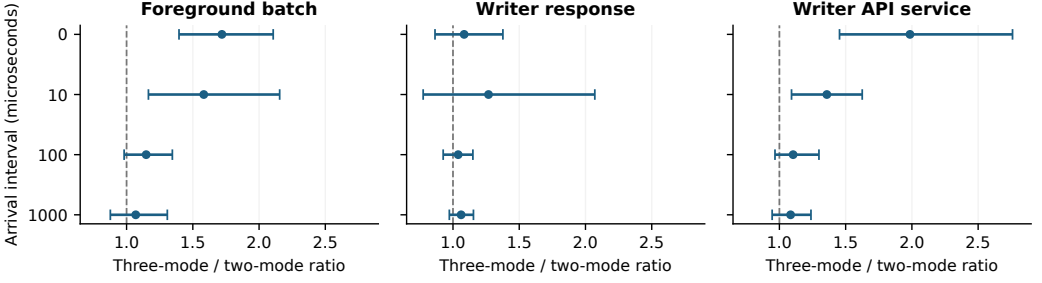


Fig. 2. New shared-cap timing comparison, $r = 2$. Points are geometric means of eight paired block ratios of process medians; bars are 95% percentile intervals from 10,000 fixed block-bootstrap resamples. Ratios above 1 favor two modes. Writer response is intended arrival to return; service is invocation to return. Each batch contributes its median across 64 writer requests. All four intervals and three metrics are shown.

Earlier timings and stronger operations. Earlier three-mode foreground medians improve in four settings and worsen in seven. Shared- $r = 2$ two/three modes use 10,802/10,786 calls and 7,131/3,641 protected transformations; three modes raise total transformations to 14,427. Rebase retains 4,219/4,528 protected merges at $r = 0/2$. All 1,728 batch-content arrival units give mixed batch/sequential ratios: 0.906–1.189 foreground, 0.511–1.404 background p95 and 0.917–1.281 protected spans. A document-state guard enables four new reuse rows among 96 while preserving newer references/diagnostics and all 288 common snapshots. Its timing is unmeasured; merges remain protected.

Comparing shared and per-update contracts. The local baseline resets retries before each update in unchanged patch04. Eight arms cover unmodified baseline, instrumented original, shared two/three at $r = 2$ and local two/three at $r = 1, 2$. Eight cyclic Latin blocks balance process positions. Each process serves 32 serial updates and 64 FIFO writers with precomputed texts, no supplied budget, and intervals $d \in \{0, 10, 100, 1000\} \mu s$. Two warm-up and ten measured batches per interval are shuffled before execution. All 2,560 measured and 512 warm-up units pass source checks; 12 corruptions reject. Arrivals are authored.

For actual update writes B_i , local modes satisfy $Q_i = 1 + f_i$, $f_i \leq \min(B_i, r)$, and inside count at most $1[B_i \geq r]$ (two) or $1[B_i \geq r + 1]$ (three). Every update and batch meets the archived count bounds. At 100 μs , local three-mode $r = 2$ has median 38 calls and zero protected transformations, versus shared $r = 2$'s 34 and six; their universal caps differ (96/34). Full four-interval count tables remain archived. Foreground-period writes depend on completion time; reconstruction checks 7,392 earlier batches without establishing a common controlled writer budget.

Shared three/two foreground ratios at 0/10 μs are 1.72 [1.40, 2.11] and 1.58 [1.17, 2.16]; both slower-arrival intervals and all writer-response intervals include 1. Counter snapshots lie inside foreground duration. Original/baseline ratios are 0.83–1.08 with all intervals including 1, leaving instrumentation overhead unidentified. Intervals are descriptive and multiplicity-unadjusted; eight blocks limit between-process resolution, and resampling cannot remove host/order effects. Native execution took 237.84 seconds; setup/build and analysis costs are archived.

9 Related Work and Research Lineage

Fallback and online contention management. Kung and Robinson [5], Section 3.3/p.220, retain the critical-section semaphore during reexecution after repeated validation failure. Retry caps in lock elision [22, 23] and RPC systems [27, 28] also predate ours. HTM aborts need not consume

writes [24]; RPC budgets may cap rates/concurrency and permit failure. Neither supplies this persistent-job resource contract.

Welc et al. [29], Sections 3.1/7.1, preserve and protect a transaction prefix during irrevocable transition; failed token acquisition leaves their second-abort heuristic revocable. Attiya et al. [20], Sections 2–3/5, analyze an unaware Greedy manager with an $O(ks)$ makespan guarantee and deterministic lower bounds for per-job failures k and resources s . Alistarh et al. [21], Sections 4–5/7, optimize abort timing with known abort cost, unknown remaining duration and later probabilistic progress. Unknown disturbances, prefix reuse and all-policy analysis are inherited; their guarantees differ from global external writes and exact persistent-job work.

Guerraoui et al. [3], Sections 3–4, give bounded-delay progress and competitive makespan for priority-based abort/wait management. Diegues and Romano [4], Sections 4–6 and errata, tune attempt budgets from per-block GCC measurements. These progress/cycle results do not calibrate our kernel-work model.

Synchronization synthesis and robust control. Černý et al. [11] choose retries, guards and caches in partial concurrent programs with a scheduler and weighted limit-average safety objective. Their Theorem 6 uses quantitative probabilistic bisimulation; the erratum corrects complexity to PSPACE. Later work optimizes protected-statement counts and statistically profiles acquisition/contention [12]; preemption-safety synthesis also addresses deadlock-free lock placement [13]. Finite n, B, r instances admit general-game encodings; our archived initial-total-cost potential proves neither source equivalence nor intermediate-state quotients.

Mannor et al. [8] supply global-deviation-budget DP; Postek et al. [17] give finite-candidate DP and causal budgeted routing. Information-state minimax control [25] also precedes our recurrence. Our new specialization is resource domination of arbitrary retained-preparation programs by finite trees and its exact W/L values. Backward induction and Pareto composition are inherited. The earlier repair-interface information-state construction remains in the supplement.

Scheduling with testing and explorable uncertainty. In the canonical $r = 0$ game, fresh completion is untested execution with cost $u_i = w_i$. Cached completion combines a test of cost $t_i = w_i$ with execution of cost $p_i \in \{0, w_i\}$. Thus W counts test plus execution work and L execution work alone. For a fixed deterministic tree, a realized mismatch set H , $|H| \leq B$, can be fixed as hidden bad-job labels and reproduces that path; conversely own-gate writes realize those labels. Dynamic writing alone therefore does not distinguish this canonical game from testing.

Dürr et al. [31], Sections 1.2/2/6, introduce the adversarial testing model with unit tests, possibly deferred execution, and competitive completion-time/makespan objectives. Binary zero/upper-bound outcomes are already present. Buld and Schulz [34], Sections 1.1/2.1 and Theorem 3.2, allow job-dependent test times and upper bounds, accommodating our operation-level parameters; their job weights scale completion-time importance, whereas ours are kernel work. Neither result supplies our cardinality-bounded bad comparisons and joint W/L cap.

Damerius et al. [32], Section 2 and Appendix D, already separate testing expenditure from execution and impose a *policy-side* testing budget. Our B bounds adversarial bad outcomes, not expenditure: with $B = 0$ preparation is still permitted. Their offline comparator knows processing times; our informed policy knows only B and remains causal. Dufossé et al. [33], Sections 2–4/6, explicitly study adaptive testing and minimax game trees. Their oracle reveals actual short/long duration, even for untested execution, and optimizes competitive total completion time; its polynomial adaptive result is conditional on the two-phase conjecture. Their state grid is not an input precedence DAG.

Albers and Eckl [35], Section 1.1, already allow unequal tests and deferred execution for competitive completion-time/makespan objectives. Albers and Janke [36], Sections 1–2, bound failing jobs

by Γ and aggregate largest adverse processing times for irrevocable online assignment to parallel machines. Cardinality-bounded adverse work is inherited; our specialization returns conditional-completion policies for separate absolute W/L caps and retry allowance r .

Lawler [30] already solves the precedence-constrained maximum completion-cost objective by selecting an eligible last job backward. We use that algorithm and exchange principle unchanged. Theorem 6’s claim is the equivalence from all admissible completion programs under a W cap to the induced scheduling instance. Without the L constraint, all-fresh already minimizes $W = \Omega$; the joint constraint is essential.

Runtime adaptation and partial reuse. Lim and Agarwal [37], Sections 3.2–3.5, use contention estimates and hysteresis to switch synchronization; Usui et al. [38], Section 2.2, compare contention, retry and transactional overhead. Such cost-benefit adaptation predates ours. Universal restart policies [39] compare independent Las Vegas runs in expected time. Build scheduling [1], self-adjusting cost semantics [2] and demand-driven incremental computation [26] reuse unaffected work. Those stronger operations can violate the whole-kernel premise; the supplement preserves our separate footprint-repair results.

10 Limitations and Threats to Validity

The whole-kernel contract excludes replenishment, structural change, exceptions, external completion and retained guards. In an earlier charged scalar objective, retained guards improve five of 384 inputs (e.g., 242 versus 243), and one intermediate state improves from 45 to 43. Those are stronger-interface counterexamples for that objective, not evaluations of the present W/L frontier. Optimality for arbitrary lock APIs is unclaimed.

Results require deterministic policies, fixed positive whole-kernel work and a start before computed records. General- r polynomial attainment covers independent jobs or $r \geq B$ DAGs; Lawler and the exact hidden-comparison frontier use $r = 0$. Finite general synthesis may be exponential, and the positive-retry unknown-budget least curve remains unresolved. Numeric IDs or timing channels change observability. A supplied B is a premise. Roslyn checks cover selected paths, not arbitrary concurrent equivalence or numerical work transfer; rebase costs remain uncalibrated.

One-workstation authored arrivals, instrumentation and cache/JIT effects limit timing inference; human benefit is unmeasured. Earlier scalar gains occur in 2/96 cells, cost fits succeed in 2/8, and all-budget compilation can exceed requested-value costs. In all 16 Valkey digest cells, cooperating writer-side maintenance beats compiled reader repair in time/transfer, with mixed maximum server-command times. This stronger operation supplies no importance evidence for our whole-kernel interface. Finite/AI-assisted checks are neither mechanical proofs nor independent reproduction.

11 Conclusion

Independent jobs admit exact polynomial joint resource frontiers. Enough retries remove precedence and cached-observation dependence from supplied-budget optimization; without spare calls, both can affect total work. A full-program domination proof supports finite synthesis elsewhere. Java checks the operation contract; Roslyn preserves selected behavior with mixed timings.

Data Availability

The supplement [19] provides code, inputs, all outcomes/corrections, derivations and replay commands; prior charged/partial-repair results and fixed163 remain preserved. Zero-fee evidence [18] is unchanged. Hashes bind projected files. Modified sources retain notices; upstream archives, runtimes, articles and private correspondence are excluded.

References

- [1] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems à la Carte. *PACMPL* 2, ICFP, Article 79, 1–29. <https://doi.org/10.1145/3236774>. Author version: <https://simon.peytonjones.org/assets/pdfs/build-systems-original.pdf>.
- [2] Ruy Ley-Wild, Umut A. Acar, and Matthew Fluet. 2009. A Cost Semantics for Self-Adjusting Computation. In *POPL*, 186–199. <https://doi.org/10.1145/1480881.1480907>. Author version: <https://ttic.uchicago.edu/~pl/sa-sml/popl09.pdf>.
- [3] Rachid Guerraoui, Maurice Herlihy, and Bastian Pochon. 2005. Toward a Theory of Transactional Contention Managers. In *PODC*. Author version. <https://lpdwww.epfl.ch/upload/documents/publications/neg--509904679f181-pochon.pdf>.
- [4] Nuno Diegues and Paolo Romano. 2014. Self-Tuning Intel Transactional Synchronization Extensions. In *ICAC*, 209–219. <https://www.usenix.org/conference/icac14/technical-sessions/presentation/diegues>. Errata.
- [5] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. <https://doi.org/10.1145/319566.319567>.
- [6] .NET Foundation. 2025. *Roslyn source*, commit 6c4a46a31302167b425d5e0a31ea83c9a9aa1d09. Workspace.cs, Solution.cs, SolutionCompilationState.cs and SolutionState.cs. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/tree/6c4a46a31302167b425d5e0a31ea83c9a9aa1d09>.
- [7] .NET Foundation contributors. 2020. Add SetCurrentSolution with solution transformation, pull request 42228. Source review on replayability and event ordering. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/pull/42228>.
- [8] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [9] Awi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [10] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [11] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [12] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*. <https://arxiv.org/abs/1511.07163>.
- [13] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2017. From non-preemptive to preemptive scheduling using synchronization synthesis. *Formal Methods in System Design* 50, 97–139. <https://doi.org/10.1007/s10703-016-0256-5>.
- [14] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [15] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [16] OpenJDK. 2026. ConcurrentHashMap.java, jdk17u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.
- [17] Krzysztof Postek, Ward Romeijnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. Online appendix; author code.
- [18] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e0110. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbbf422390ad8d29785ad0023f>.
- [19] Anonymous Authors. 2026. *Retry Synthesis with Bounded Calls and Protected Work: Research Artifact*. General-retry and native-wrapper supplement, commit 9081205. <https://github.com/research-artifact-archive/research-artifact/tree/90812054158ced9ae313240f9211ec5f61f51fe>.
- [20] Hagit Attiya, Leah Epstein, Hadas Shachnai, and Tami Tamir. 2006. Transactional Contention Management as a Non-Clairvoyant Scheduling Problem. In *PODC*. <https://cs.idc.ac.il/~tami/Papers/podc06.pdf>.
- [21] Dan Alistarh, Syed Kamran Haider, Raphael Kübler, and Giorgi Nadiradze. 2018. The Transactional Conflict Problem. *arXiv preprint arXiv:1804.00947v1*. <https://arxiv.org/abs/1804.00947>.
- [22] Free Software Foundation. 2025. GNU C Library 2.42: x86 lock elision, revision d2097651. elision-lock.c and elision-conf.c. <https://github.com/bminor/glibc/tree/d2097651cc57834dbfcaa102ddfaca0d86cfb66/sysdeps/unix/sysv/linux/x86>. Official tunables: https://www.gnu.org/software/libc/manual/html_node/Elision-Tunables.html.

- [23] OpenJDK. 2025. HotSpot 21.0.8+9 RTM monitor code, revision 54f20959. `c2_MacroAssembler_x86.cpp` and `globals_x86.hpp`. <https://github.com/openjdk/jdk21u/tree/54f2095960f01d957f2335cafa0defb956e13a2c/src/hotspot/cpu/x86>.
- [24] GNU Compiler Collection. n.d. x86 Transactional Memory Intrinsics. Accessed September 10, 2026. <https://gcc.gnu.org/onlinedocs/gcc/x86-transactional-memory-intrinsics.html>.
- [25] Aditya Dave, Nishanth Venkatesh, and Andreas A. Malikopoulos. 2024. Approximate Information States for Worst-Case Control and Learning in Uncertain Systems. *arXiv preprint* arXiv:2301.05089v2. <https://arxiv.org/abs/2301.05089>.
- [26] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. 2014. Adapton: Composable, Demand-Driven Incremental Computation. In *PLDI*. <https://doi.org/10.1145/2594291.2594324>. Author version: <https://www.cs.tufts.edu/~jfooster/papers/pldi14.pdf>.
- [27] Mike Ulrich. 2016. Addressing Cascading Failures. In *Site Reliability Engineering*, Chapter 22, Retries. <https://sre.google/sre-book/addressing-cascading-failures/>.
- [28] Envoy contributors. n.d. CircuitBreakers.Thresholds.RetryBudget. Accessed September 10, 2026. https://www.envoyproxy.io/docs/envoy/latest/api-v3/config/cluster/v3/circuit_breaker.proto.html.
- [29] Adam Welc, Bratin Saha, and Ali-Reza Adl-Tabatabai. 2008. Irrevocable Transactions and their Applications. In *SPAA*, 285–296. <https://doi.org/10.1145/1378533.1378584>.
- [30] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. <https://doi.org/10.1287/mnsc.19.5.544>. Author technical report ERL-M327 (1971): <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1971/Archive/ERL-m-327.pdf>.
- [31] Christoph Dürr, Thomas Erlebach, Nicole Megow, and Julie Meißner. 2020. An Adversarial Model for Scheduling with Testing. Author preprint arXiv:1709.02592v3. <https://arxiv.org/abs/1709.02592v3>.
- [32] Christoph Damerius, Peter Kling, Minming Li, Chenyang Xu, and Ruilong Zhang. 2023. Scheduling with a Limited Testing Budget: Tight Results for the Offline and Oblivious Settings. Author preprint arXiv:2306.15597v1. <https://arxiv.org/abs/2306.15597v1>.
- [33] Fanny Dufossé, Christoph Dürr, Noël Nadal, Denis Trystram, and Óscar C. Vázquez. 2021. Scheduling with a Processing Time Oracle. Author preprint arXiv:2005.03394v3. <https://arxiv.org/abs/2005.03394v3>.
- [34] Felix Buld and Andreas S. Schulz. 2026. Scheduling with Testing: Competitive Algorithms for Minimizing the Total Weighted Completion Time in the Adversarial Model. Author preprint arXiv:2606.25166v1. <https://arxiv.org/abs/2606.25166v1>.
- [35] Susanne Albers and Alexander Eckl. 2020. Explorable Uncertainty in Scheduling with Non-Uniform Testing Times. Author preprint arXiv:2009.13316. <https://arxiv.org/abs/2009.13316>.
- [36] Susanne Albers and Maximilian Janke. 2021. Online Makespan Minimization with Budgeted Uncertainty. In *WADS*. https://doi.org/10.1007/978-3-030-83508-8_4. Author version: <https://maximilian-janke.de/data/BudgetedUncertainty.pdf>.
- [37] Beng-Hong Lim and Anant Agarwal. 1994. Reactive Synchronization Algorithms for Multiprocessors. In *ASPLOS VI*, 25–35. Author version: <https://groups.csail.mit.edu/cag/pub/papers/pdf/reactive.pdf>.
- [38] Takayuki Usui, Reimer Behrendts, Jacob Evans, and Yannis Smaragdakis. 2010. Adaptive Locks: Combining Transactions and Locks for Efficient Concurrency. *Journal of Parallel and Distributed Computing*. <https://doi.org/10.1016/j.jpdc.2010.02.006>. Author version: <https://people.cs.umass.edu/~yannis/adaptive-jpdc.pdf>.
- [39] Michael Luby, Alistair Sinclair, and David Zuckerman. 1993. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters* 47, 4, 173–180. Author version: <https://www.cs.utexas.edu/~diz/pubs/speedup.pdf>.